discussions. If the standard deviation is zero or $\boldsymbol{x} = \boldsymbol{\mu}$, then LayerNorm outputs a vector with all its components equal to zero.

2.1 The Uniform Vector and the Mean Vector

The vector definition of layer normalization in equation 4 requires us to define two new vectors: the *uniform vector* and the *mean vector*. Since these are non-standard vectors, it is important to understand the properties of these vectors.

The *uniform vector* or $\mathbf{1} = [1, 1, 1, 1, \cdots 1]^T$, is called so because all its components are equal or uniform and set to 1. This vector plays a very important role in the formulation and understanding of layer normalization. An important thing to note here is the norm of the uniform vector: $\|\mathbf{1}\|_2 = \sqrt{\mathbf{1}^T\mathbf{1}} = \sqrt{d}$.

The *mean vector* in the context of LayerNorm is defined in a non-traditional manner. Traditionally, a mean vector is the sum of a few vectors. But in our definition, the mean vector, $\boldsymbol{\mu} = \mu\mathbf{1}$, is a scaled version of the uniform vector. It is scaled by a value that is the mean or the average of all the components of the hidden vector and is oriented in the direction of the uniform vector. An interesting property of the mean vector is its norm. Let's define θ_{x1} as the angle between vectors $\boldsymbol{x}$ and the uniform vector. Then taking the L2 norm of the mean vector:

$$\begin{aligned}\|\boldsymbol{\mu}\|_2 = \|\mu\mathbf{1}\|_2 = \mu\|\mathbf{1}\|_2 = \left(\frac{1}{d}\mathbf{1}^T\boldsymbol{x}\right)\|\mathbf{1}\|_2 \\ = \frac{1}{d}\|\mathbf{1}\|_2^2\|\boldsymbol{x}\|_2\cos\theta_{x1} = \|\boldsymbol{x}\|_2\cos\theta_{x1}\end{aligned} \tag{5}$$

Here we replace the formula for the mean from equation 1, expand the inner product between the uniform vector and $\boldsymbol{x}$ in terms of their norms and the angle between them, and use the fact that $\|\mathbf{1}\|_2 = \sqrt{d}$.

The norm of the *mean vector* gives us very important insights about what's going on in the layer normalization process. Equation 5 shows that the norm of the mean vector is nothing but **the projection of the underlying vector $\boldsymbol{x}$ along the uniform vector**. By definition, the mean vector, $\boldsymbol{\mu} = \mu\mathbf{1}$, is oriented along the direction of the uniform vector. In other words, given that θ is the angle between vectors $\boldsymbol{x}$ and the uniform vector,

$$\|\boldsymbol{\mu}\|_2 = \|\boldsymbol{x}\|_2\cos\theta_{x1} = \frac{\boldsymbol{x}\cdot\mathbf{1}}{\|\mathbf{1}\|_2} = \boldsymbol{x}\cdot\hat{\mathbf{1}} \tag{6}$$

and

$$\boldsymbol{\mu} = (\boldsymbol{x}.\hat{\mathbf{1}})\hat{\mathbf{1}} \tag{7}$$

Note that $\hat{\mathbf{1}}$ is the unit vector corresponding to the uniform vector $\mathbf{1}$. To the best of our knowledge, the geometrical meaning of the mean vector has not been discussed in literature before our work. In the next section, we incorporate this information to explain the LayerNorm process, giving a very intuitive and informative description of the process.

2.2 Explanation of Layer Normalization

If we go through the steps of layer normalization as discussed in the beginning of section 2, the component-wise subtraction of the mean of all components of a vector can now be written completely in terms of the vector $\boldsymbol{x}$. We define $\boldsymbol{x}_\perp^{\mathbf{1}}$ as the component of $\boldsymbol{x}$ orthogonal to the uniform vector as follows:

$$\boldsymbol{x}_\perp^{\mathbf{1}} = \boldsymbol{x} - \boldsymbol{\mu} = \boldsymbol{x} - (\boldsymbol{x}.\hat{\mathbf{1}})\hat{\mathbf{1}} \tag{8}$$

Thus, subtracting the mean of the components of a vector is the same as the removal of the projection of the vector along the uniform vector. The complete standardization step of the layer normalization operation can now be written as:

$$\boldsymbol{y} = \sqrt{d}\frac{\boldsymbol{x}_\perp^{\mathbf{1}}}{\|\boldsymbol{x}_\perp^{\mathbf{1}}\|_2} \tag{9}$$

where d is the dimensionality of the vector being normalized. This shows that **layer normalization can be simply defined as the normalization of the component of a vector orthogonal to the uniform vector, accompanied by a scaling factor**. With this, we present the simple and elegant definition of the layer normalization process with a deep geometric meaning. The most intuitive recipe of the layer normalization process is shown in Algorithm 1.

Algorithm 1 : The Standardization Step in Layer Nor

1: Throw away the component of a vector along the uniform vector, $\mathbf{1} = [1, 1, 1, 1, \cdots 1]^T$
2: Normalize the remaining vector
3: Scale the resulting vector by $\sqrt{d}$, where d is the dimensionality of the vector space

Equation 9 also shows that the norm of the standardized vector is $\sqrt{d}$ and is oriented orthogonal to

the direction of the uniform vector, which means it exists in a subspace orthogonal to the uniform vector.

2.3 The Irreversibility of Layer Normalization

The idea of reversibility is discussed briefly while introducing batch normalization (Ioffe and Szegedy, 2015), which normalizes each feature dimension in the input vector $\boldsymbol{x}$ independently by calculating the mean and standard deviation statistics over a large sample of such vectors. Specifically, let $\boldsymbol{x}^1, \boldsymbol{x}^2, \ldots \boldsymbol{x}^b$ be a set of b vectors in the training set, each vector of dimension d. Then, both the batch normalization and layer normalization processes for each component can be represented in the following two-step process:

$$y_i = \frac{x_i - \mathbb{E}[x_i]}{\sqrt{Var(x_i)}} \text{ (standardization)} \quad (10)$$

$$z_i = \alpha_i x_i + \beta_i \text{ (scale-and-shift)} \quad (11)$$

The key difference between batch normalization and layer normalization lies in how the expectation values and variance are calculated for each component. In the case of batch normalization, the mean and variance for each dimension are calculated over the training set. This means that $\mathbb{E}[x_i]$ and $Var(x_i)$ are the same for each vector $\boldsymbol{x}^j \in \{\boldsymbol{x}^1, \boldsymbol{x}^2, \ldots \boldsymbol{x}^b\}$, but different for each component of the vector. In total, for a d-dimensional hidden vector space, there are $2d$ statistics in batch normalization, d for the mean and d for variance. Due to this, the information lost during the standardization step in batch normalization, if important, can be recovered in the scale-and-shift step by simply learning $\alpha_i = \sqrt{Var(x_i)}$ and $\beta_i = \mathbb{E}[x_i]$, since we have $2d$ learnable parameters in batch normalization. The scale-and-shift step was a very conscious design choice of the inventors of batch normalization to allow for batch normalization to represent an identity transformation if the original hidden representations were optimal for the network.

A subtle but very important difference with layer normalization is how the expectation and variances are calculated for each component. For layer normalization, these statistics are calculated for each vector independently. $\mathbb{E}[x_i]$ and $Var(x_i)$ in layer normalization are the same for each component x_i within a vector but are different for different vectors. This means that two variables are created in LayerNorm per hidden vector. As LLMs will normalize way more than d vectors throughout the course of their training, thus creating more than $2d$ statistics, the number of statistic variables used in LayerNorm is very high. Thus, the information lost during layer normalization cannot be recovered by learning just $2d$ components of the $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ learnable parameters. This shows that **information once lost during layer normalization cannot be recovered during the scale-and-shift step, making the layer normalization process irreversible**.

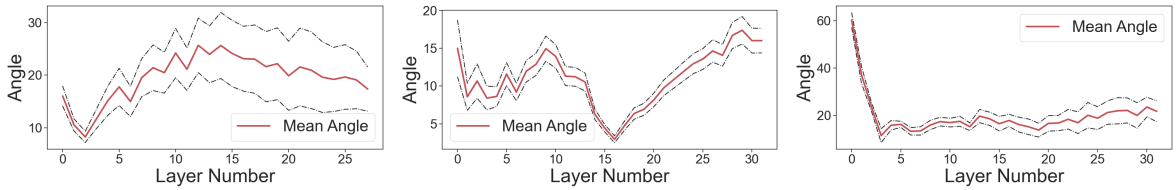
Model Name	Model Dim (d)	Num Params	Num Layers	Norm. Type
GPT-2 XL	1600	1.5B	48	Layer
GPT-Neo 1.3B	2048	1.3B	24	Layer
Pythia-1.4B	2048	1.4B	24	Layer
GPT-J 6B	4096	6.0B	28	Layer
Pythia-6.9B	4096	6.9B	32	Layer
Llama-2-7B	4096	7.0B	32	RMS
Llama-3-8B	4096	8.0B	32	RMS

Table 1: List of models used in experiments.

3 Experiments

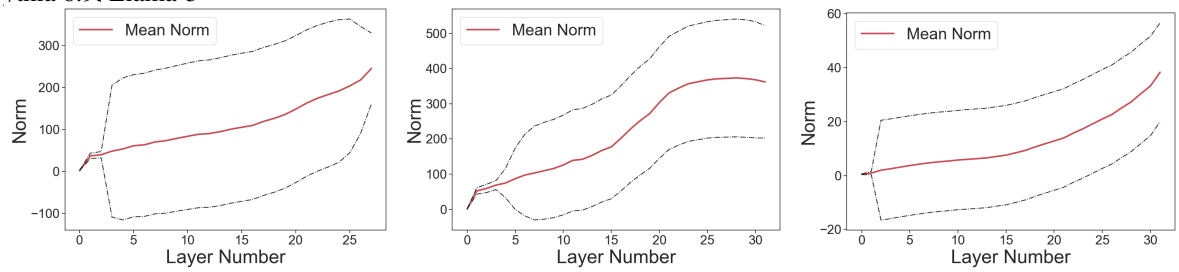
After a detailed theoretical analysis of the effects of layer normalization on a vector, we next study how LayerNorm affects the hidden representations of LLMs in practice. While prior works have studied the effects of LayerNorm on downstream model performance and training convergence (Ba et al., 2016; Xu et al., 2019; Zhang and Sennrich, 2019), there has been a surprising gap in the literature about studying the effect of LayerNorm on the internal representations of a model at inference time. To do so, we pass one million tokens from Wikipedia articles through various models and study how LayerNorm modifies a hidden vector in the representation space.

Methods and Models We study the impact of LayerNorm on the hidden representations of 7 decoder-only LLMs across two size categories. The models used are listed in Table 1. GPT2XL, GPT-Neo-1.3B, and Pythia-1.4B represent the small LLM size category, whereas GPT-J-6B, Pythia-6.9B, Llama-2-7B, and Llama-3-8B represent the medium LLM size category. We pass one million tokens from Wikipedia articles through each model and capture the hidden representations for all tokens before and after each normalization layer. This is done for each layer inside the model, which creates many terabytes of data to be analyzed.

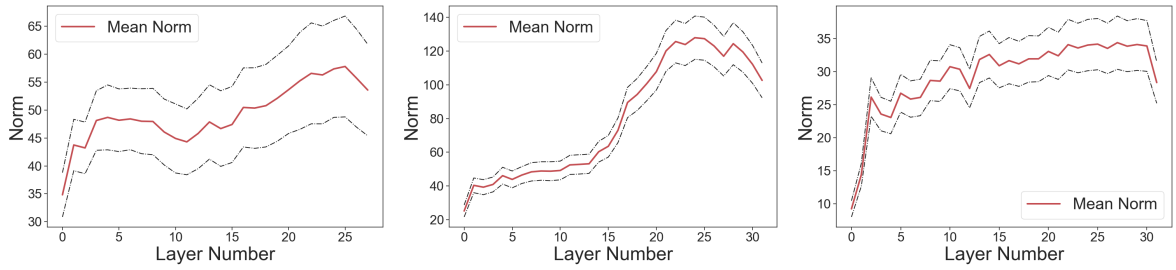


(a) GPT-J (b) Pythia 6.9 (c) Llama-3

Figure 3: Rotation angle (in degrees) between the hidden vectors and Post-LN1 vectors across all layers for GPT-J, Pythia 6.9, Llama-3



(a) Residual Stream Pre-LN1 GPT-J (b) Residual Stream Pre-LN1 Pythia 6.9 (c) Residual Stream Pre-LN1 Llama-3



(d) Residual Stream Post-LN1 GPT-J (e) Residual Stream Post-LN1 Pythia 6.9 (f) Residual Stream Post-LN1 Llama-3

Figure 4: This figure shows the growing norm of the residual stream or hidden vectors at each layer (a-c) and how LayerNorm and RMSNorm regulate the growing norms (d-f) for GPTJ 6B, Pythia 6.9B and Llama3 8B. The dashed lines in the line plots represent one standard deviation.

Norms were calculated using the L2 norm, and angle differences in degrees were determined through cosine similarity calculations to measure the orientation changes induced by normalization.

As outlined in Table 1, we also use two types of normalization methods. Models like GPT-2 XL, GPT-Neo 1.3B, Pythia-1.4B, GPT-J 6B, and Pythia-6.9B employ LayerNorm (Ba et al., 2016), whereas Llama-2-7B and Llama-3-8B utilize RMSNorm (Zhang and Sennrich, 2019). The crucial difference between LayerNorm and RMSNorm is the subtraction of the mean vector (Figure 1). In this paper, we study how LayerNorm and RMSNorm operate on the hidden representations.

3.1 Norm Stabilization

As a token representation passes through the different layers inside an LLM, the hidden representations accumulate due to residual connections (He et al., 2016). These hidden representations get added to the residual stream at each layer as shown in equation 17 and can cause the norm of the hidden vectors to grow. In fact, the norm of the hidden vectors at each layer grows disproportionately with large standard deviations, as can be seen in Figure 4 (a-c). The growing norm of the hidden vectors is extreme for some models more than others, as is seen for GPT-J and Pythia-6.9B. The growing norm of hidden representations for the remaining models from Table 1 and the effect of layer normalization can be seen in Figure 7 (appendix).

The significant role of LayerNorm and RMSNorm in stabilizing the growing norm of the hidden vectors at each layer can be seen in Figure 4 (d-f). The standardization step in layer normalization first modifies the norm of each vector to $\sqrt{d}$, where d is the dimensionality of the representation space. The norms of the standardized vectors are further modified marginally by the scale-and-shift steps. Layer normalization affects both the mean and the spread of the norms, where the standard deviations in the norm are reduced by factors be-